

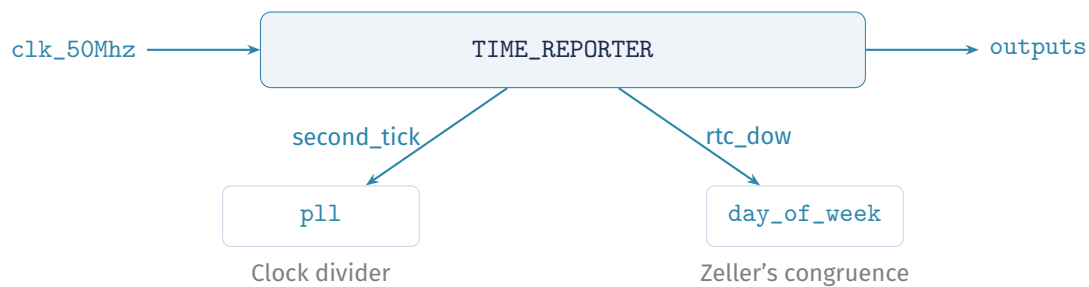
Contents

1	Overview	2
1.1	Sub-module Hierarchy	2
2	Port List	2
2.1	Outputs	2
2.2	Inputs	2
3	Register Map	3
3.1	RTC Registers	3
3.2	Alarm Registers	3
3.3	Timer Registers	3
4	Functional Description	3
4.1	Clock Divider (p11)	3
4.2	Timekeeping and Rollover	4
4.2.1	Leap-year Logic	4
4.3	Input Validation	4
4.4	Alarm	5
4.5	Countdown Timer	5
4.6	Day of Week (day_of_week)	5
5	Write / Tick Priority	5
6	Reset State	6
7	Timing Notes	6
8	Instantiation Example	7
9	File List	7

1 Overview

TIME_REPORTER is a synthesisable Verilog RTC module designed for 50 MHz FPGA operation. It maintains a full Gregorian calendar date and time, provides a day-of-week output computed via Zeller's congruence, and exposes alarm and countdown-timer peripherals — all accessible through a simple address-mapped register interface.

1.1 Sub-module Hierarchy



- `pll` — divides the 50 MHz clock to produce one `second_tick` pulse per second.
- `TIME_REPORTER` — main RTC logic: timekeeping, register writes, input validation, alarm, and timer.
- `day_of_week` — combinational Zeller's congruence calculator, output encoding: 0=Sun, 1=Mon, ..., 6=Sat.

2 Port List

2.1 Outputs

Signal	Dir	Width	Description
<code>rtc_year</code>	Out	[13:0]	Current year (1990–9999).
<code>rtc_month</code>	Out	[3:0]	Current month (1–12).
<code>rtc_day</code>	Out	[4:0]	Day of month (1–31).
<code>rtc_hour</code>	Out	[4:0]	Hour in 24 h format (0–23).
<code>rtc_minute</code>	Out	[5:0]	Minute (0–59).
<code>rtc_second</code>	Out	[5:0]	Second (0–59).
<code>rtc_dow</code>	Out	[2:0]	Day of week, combinational. 0=Sun ... 6=Sat.
<code>alarm_out</code>	Out	1	High when RTC matches all alarm registers and <code>alarm_en=1</code> .
<code>timer_out</code>	Out	1	High when countdown reaches 00:00:00 and <code>timer_en=1</code> .
<code>error</code>	Out	1	Pulses high for one cycle on any invalid register write.

2.2 Inputs

Signal	Dir	Width	Description
<code>clk_50Mhz</code>	In	1	50 MHz system clock.
<code>rst_n</code>	In	1	Active-low asynchronous reset.

Signal	Dir	Width	Description
wr_en	In	1	Write enable — hold high for one cycle to commit a register write.
alarm_en	In	1	Enables alarm comparison; forces alarm_out=0 when de-asserted.
timer_en	In	1	Enables timer decrement and timer_out assertion.
data_in	In	[13:0]	Write data for the addressed register.
addr	In	[3:0]	Register address (see Section 3).

3 Register Map

Assert `wr_en=1` for exactly one rising clock edge with the target address on `addr` and the value on `data_in`. Out-of-range values are silently discarded and `error` is pulsed for one cycle.

3.1 RTC Registers

Addr	Width	Name	Description / Valid Range
0x0	[13:0]	rtc_year	Year. Range: 1990–9999.
0x1	[3:0]	rtc_month	Month. Range: 1–12.
0x2	[4:0]	rtc_day	Day. Range: 1– <i>maxday</i> (accounts for month and leap year).
0x3	[4:0]	rtc_hour	Hour. Range: 0–23.
0x4	[5:0]	rtc_minute	Minute. Range: 0–59.
0x5	[5:0]	rtc_second	Second. Range: 0–59.

3.2 Alarm Registers

Addr	Width	Name	Description / Valid Range
0x6	[13:0]	alarm_year	Alarm year. Same validation as <code>rtc_year</code> .
0x7	[3:0]	alarm_month	Alarm month.
0x8	[4:0]	alarm_day	Alarm day.
0x9	[4:0]	alarm_hour	Alarm hour.
0xA	[5:0]	alarm_minute	Alarm minute.
0xB	[5:0]	alarm_second	Alarm second.

3.3 Timer Registers

Addr	Width	Name	Description
0xC	[4:0]	timer_hour	Countdown load value — hours (0–23).
0xD	[5:0]	timer_minute	Countdown load value — minutes (0–59).
0xE	[5:0]	timer_second	Countdown load value — seconds (0–59).

4 Functional Description

4.1 Clock Divider (p11)

The `p11` module counts from 0 to `DIV – 1` (default 50 000 000) and emits a single-cycle high pulse on `tick`, producing one `second_tick` per second at 50 MHz. The `DIV` parameter can be overridden via `defparam` to accelerate simulation:

```
1 defparam dut.pll_inst.DIV = 100; // 1 tick per 100 clocks
```

Listing 1: Overriding PLL divisor in testbench

4.2 Timekeeping and Rollover

On every `second_tick` the RTC registers increment with full calendar-aware rollover. To keep the critical path flat, five combinational *carry wires* are pre-computed and then used as independent gate conditions on each field:

```
1 wire rtc_minute_ismax = (rtc_second >= 59);
2 wire rtc_hour_ismax   = rtc_minute_ismax && (rtc_minute >= 59);
3 wire rtc_day_ismax    = rtc_hour_ismax   && (rtc_hour   >= 23);
4 wire rtc_month_ismax  = rtc_day_ismax    && (rtc_day    >= maxday);
5 wire rtc_year_ismax   = rtc_month_ismax  && (rtc_month  >= 12);
```

Listing 2: Rollover carry wires

Each field is then updated independently and in parallel:

```
1 rtc_second <= (rtc_second >= 59) ? 6'd0 : rtc_second + 1;
2 if (rtc_minute_ismax)
3     rtc_minute <= (rtc_minute >= 59) ? 6'd0 : rtc_minute + 1;
4 if (rtc_hour_ismax)
5     rtc_hour <= (rtc_hour >= 23) ? 5'd0 : rtc_hour + 1;
6 if (rtc_day_ismax)
7     rtc_day <= (rtc_day >= maxday) ? 5'd1 : rtc_day + 1;
8 if (rtc_month_ismax)
9     rtc_month <= (rtc_month >= 12) ? 4'd1 : rtc_month + 1;
10 if (rtc_year_ismax)
11     rtc_year <= rtc_year + 1;
```

Listing 3: Flat parallel field updates on `second_tick`

Why this matters: the original nested-if structure created a ≈ 28 ns path from `rtc_year[9]` to `rtc_year[1]` — 8 ns over the 20 ns budget. The flat approach reduces that to a single carry-wire (1 FF) feeding a 2:1 mux and adder.

4.2.1 Leap-year Logic

`maxday` is derived combinatorially from `rtc_month` with Gregorian leap-year rules:

$$\text{leap} = (\text{year}[1:0] = 0 \wedge \text{year} \bmod 100 \neq 0) \vee (\text{year} \bmod 400 = 0)$$

February's `maxday` is 29 when `leap` is true, 28 otherwise.

4.3 Input Validation

All register writes pass through a combinational validator. Out-of-range data sets `invalid_data_flag`, which prevents the write and pulses `error` for one cycle.

Register	Min	Max
rtc_year / alarm_year	1990	9999
rtc_month / alarm_month	1	12
rtc_day / alarm_day	1	<i>maxday</i> (month-aware)
rtc_hour / alarm_hour	0	23
rtc_minute / alarm_minute	0	59
rtc_second / alarm_second	0	59

4.4 Alarm

alarm_out is a combinational output that goes high when alarm_en=1 and all six alarm registers exactly match their RTC counterparts. It de-asserts automatically on the next second_tick when any field diverges.

Timing note: the six-field comparison (14+4+5+5+6+6 bits) is on the combinational output path. If alarm_out is timing-critical, register it — the one-cycle latency is negligible for alarm use cases.

4.5 Countdown Timer

The timer is a loadable HH:MM:SS down-counter. When timer_en=1, on each second_tick the timer decrements by one second with borrow propagation (seconds borrow from minutes, minutes from hours). timer_out asserts combinatorially when timer_en=1 and the timer value is exactly 00:00:00. The timer holds at zero — it does not auto-reload.

4.6 Day of Week (day_of_week)

Zeller's congruence computes rtc_dow purely combinatorially from the registered rtc_year, rtc_month, rtc_day:

$$h = \left(d + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + k + \left\lfloor \frac{k}{4} \right\rfloor + \left\lfloor \frac{j}{4} \right\rfloor + 5j \right) \bmod 7$$

where d = day, m = adjusted month (Jan/Feb → 13/14 of prior year), $k = y \bmod 100$, $j = \lfloor y/100 \rfloor$. The raw result h (0=Sat, 1=Sun) is remapped to 0=Sun ... 6=Sat via $(h + 6) \bmod 7$.

5 Write / Tick Priority

The main sequential always block implements a strict three-way priority on every rising clock edge:

Reset (`rst_n=0`): all RTC and alarm registers → reset values.

Register write (`wr_en=1`): write takes priority; simultaneous tick is skipped.

Tick (`second_tick`): RTC and timer registers increment.

6 Reset State

Register	Reset Value
<code>rtc_year</code>	1990
<code>rtc_month</code>	1 (January)
<code>rtc_day</code>	1
<code>rtc_hour</code>	0
<code>rtc_minute</code>	0
<code>rtc_second</code>	0
<code>alarm_year</code>	1990
<code>alarm_month</code>	1
<code>alarm_day</code>	1
<code>alarm_h,m,s</code>	0
<code>error</code>	0

Timer registers (`timer_hour`, `timer_minute`, `timer_second`) are **not** initialised at reset. Load them via register writes before asserting `timer_en`.

7 Timing Notes

The design targets a 50 MHz clock (20 ns period). The table below summarises the paths identified during timing closure and the recommended mitigations.

Path	Root Cause	Mitigation
<code>rtc_year[9:12] → rtc_</code>	Original nested-if rollover: 5 chained comparators before the year adder (≈ 28 ns).	Flat carry-wire structure; each field gated by a single pre-computed wire.
<code>day_of_week</code> combo chain	Two <code>% /</code> operators in series (decade + century divides, then final <code>%7</code>).	Add a pipeline register between the two divide stages; <code>dow</code> is only needed once per second.
<code>alarm_out</code> comparator	Six-field combinational comparison (14+4+5+5+6+6 bits) driving an output directly.	Register <code>alarm_out</code> ; one-cycle latency is irrelevant for alarm use.
<code>pll</code> counter comparator	32-bit integer comparator against <code>CNT_MAX</code> .	Size counter to $\lceil \log_2(DIV) \rceil = 26$ bits; narrows the comparator for free.

Quartus Slow vs. Fast model: timing passing only in the *Fast 1200mV 0°C* model but failing in *Slow 1200mV 85°C* is a real violation. Silicon operates at 85°C; the slow model is the sign-off corner. The fast model passing means your logic is *close* — apply the mitigations above to close the remaining slack.

8 Instantiation Example

```

1 TIME_REPORTER u_rtc (
2     .clk_50Mhz    (clk),
3     .rst_n        (rst_n),
4     .wr_en        (reg_wr_en),
5     .alarm_en     (alarm_enable),
6     .timer_en     (timer_enable),
7     .data_in      (reg_data_in),
8     .addr         (reg_addr),
9     // RTC outputs
10    .rtc_year      (year),
11    .rtc_month     (month),
12    .rtc_day       (day),
13    .rtc_hour      (hour),
14    .rtc_minute    (minute),
15    .rtc_second    (second),
16    .rtc_dow       (dow),
17    // Interrupt outputs
18    .alarm_out     (alarm_irq),
19    .timer_out     (timer_irq),
20    .error         (rtc_error)
21 );

```

Listing 4: Instantiating TIME_REPORTER in a top-level design

9 File List

File	Description
time.v	Top-level source: pll, TIME_REPORTER, day_of_week.
tb.v	Self-checking testbench (iverilog + vvp). Overrides DIV via defparam; runs 6 test groups.